COMPLETE FYP IMPLEMENTATION GUIDE

AUTONOMOUS DRONE NAVIGATION WITH LANGUAGE-GUIDED COMMANDS

Optimized for: Ubuntu 22.04 | Core i7 | 8GB RAM

TABLE OF CONTENTS

PART 1: UNDERSTANDING THE SYSTEM

- What You Will Build
- How Everything Connects
- Technologies Explained
- System Architecture

PART 2: COMPLETE SETUP (Step-by-Step)

- Environment Setup
- All Installations
- Testing Each Component

PART 3: PROJECT STRUCTURE

- Complete Directory Map
- Where Each File Goes
- How Files Connect

PART 4: BUILDING THE SYSTEM

- Microservices Code (Complete)
- AI Models Code (Complete)
- Integration Code
- Testing Scripts

PART 5: RUNNING THE SYSTEM

- Starting All Services
- Flying the Drone
- Viewing in Grafana
- Complete Demo

PART 6: TROUBLESHOOTING

- Common Issues
- Solutions

PART 1: UNDERSTANDING THE SYSTEM

What You Will Build

You will build a **complete autonomous drone system** with these capabilities:

1. Voice/Text Commands:

```
You type: "Go to forest area and inspect for animals"

System understands → Converts to GPS coordinates → Drone flies there
```

2. Obstacle Avoidance:

```
Drone flying → Camera sees tree → YOLO detects it → RL Agent decides to avoid
→ Drone goes around
```

3. Real-Time Monitoring:

```
Drone flying → Telemetry streams → Grafana shows:

  - GPS path on map

  - Altitude graph

  - Battery level

  - Speed

  - Live camera feed
```

4. Intelligent Behavior:

```
3 AI Models work together:
```

```
  - NLP: Understands your commands
  - YOLO: Sees obstacles
  - RL Agent: Makes flying decisions
```

How Everything Connects (Simple Explanation)

Think of your system like a real company with departments:

```
┌──────────────────────────────────────────────────┐
│           YOU (The Operator)                     │
│  Type command in terminal: "Go to forest"        │
└──────────────────┬───────────────────────────────┘
                   │
                   ▼
┌──────────────────────────────────────────────────┐
│        COMMAND PARSER SERVICE                    │
│  (Microservice #1 - Uses NLP AI)                 │
│  "Go to forest" → GPS: -1.2921, 36.8219          │
└──────────────────┬───────────────────────────────┘
                   │
                   │ Publishes to Kafka Topic:
                   │ "drone.commands.flight"
                   ▼
┌──────────────────────────────────────────────────┐
│            KAFKA MESSAGE QUEUE                   │
│  (Like a post office - delivers messages)        │
│  Topic: drone.commands.flight                    │
│  Topic: drone.telemetry.gps                      │
│  Topic: drone.detections.objects                 │
└──────────┬──────────────────────┬────────────────┘
           │                      │
           │                      │
           ▼                      ▼
```

```
┌──────────────────┐   ┌────────────────────┐
│ FLIGHT CONTROL   │   │ TELEMETRY COLLECTOR  │
│ SERVICE          │   │ SERVICE              │
│ (Microservice #2)│   │ (Microservice #3)    │
│                  │   │                      │
│ Receives command │   │ Reads drone data     │
│ Sends to drone   │   │ every second         │
└────────┬─────────┘   └────────┬─────────────┘
         │                     │
         │                     │
         │                     │ Sends data to
         │                     │ InfluxDB & Kafka
         ▼                     ▼
┌──────────────────────────────────────────┐
│         PX4 SITL + GAZEBO                  │
│  (Virtual Drone + 3D Simulator)            │
│  - Receives MAVLink commands               │
│  - Sends back GPS, battery, altitude       │
│  - Provides camera feed                    │
└────────┬─────────────────────────────────┘
         │
         │ Camera Feed
         ▼
┌──────────────────────────────────────────┐
│    OBJECT DETECTION SERVICE                 │
│    (Microservice #4 - Uses YOLO AI)        │
│    Detects: trees, animals, buildings      │
└────────┬─────────────────────────────────┘
         │
         │
         │ Publishes detections to Kafka
         ▼
┌──────────────────────────────────────────┐
│    RL NAVIGATION SERVICE                    │
│    (Microservice #5 - Uses RL AI)          │
```

```
│    Reads: Target + Current position + Obstacles│
│    Decides: Go left/right/up/down to avoid   │
└──────────┬──────────────────────────────────────┘
           │
           │
           │ Sends navigation commands
           │ back to Flight Control
           └──────────► (Loop continues)


Meanwhile, all data flows to:


┌──────────────────────────────────────────────┐
│           INFLUXDB DATABASE                   │
│  Stores all telemetry with timestamps        │
└──────────┬──────────────────────────────────────┘
           │
           │
           │ Grafana queries this
           ▼
┌──────────────────────────────────────────────┐
│              GRAFANA                          │
│  YOU SEE:                                    │
│  - Live GPS position on map                  │
│  - Altitude graph going up/down              │
│  - Battery percentage                        │
│  - Radar showing drone movement              │
└──────────────────────────────────────────────┘
```

Key Concept: Microservices communicate through Kafka, not directly

Traditional way (BAD):

```
Command Parser → calls → Flight Control → calls → Telemetry
(If one service crashes, all break)
```

Microservices way (GOOD):

```
Command Parser → publishes to Kafka → Flight Control reads from Kafka

(Services are independent - if one crashes, others keep working)
```

Technologies Explained (Why Each is Used)

1. Ubuntu 22.04 Linux

What: Operating system **Why:**

- ROS 2 works best on Linux
- PX4 is developed on Linux
- Better for development than Windows
- Your i7 + 8GB RAM is perfect for Ubuntu

2. ROS 2 Humble

What: Robot Operating System **Why:**

- Connects sensors, cameras, motors
- Industry standard for robotics
- Manages communication between robot parts
- Gazebo integrates perfectly with ROS 2

Example: Camera in Gazebo → ROS 2 topic → Your Python code reads it

3. Gazebo Garden

What: 3D physics simulator **Why:**

- Simulates real drone physics (gravity, wind, propeller thrust)
- Provides 3D world (forests, buildings)
- Simulates sensors (GPS, camera, IMU)
- You see drone flying in 3D - looks real!

Without Gazebo: You just see numbers (GPS coordinates) **With Gazebo:** You SEE the drone flying over forests!

4. PX4 Autopilot (SITL Mode)

What: Drone flight controller software **Why:**

- Same software used in real drones (DJI, Pixhawk)
- SITL = Software In The Loop (runs on computer, not hardware)
- Handles low-level flight (keeping drone stable, following waypoints)

- You send high-level commands: "Go to X,Y" - PX4 handles the physics

Think of it as: You tell taxi driver "Go to airport" - driver handles steering, braking, etc.

5. MAVLink Protocol

What: Communication protocol for drones **Why:**

- Industry standard
- Commands: Takeoff, Land, Go to position
- Telemetry: GPS, battery, altitude
- Works with PX4, QGroundControl, your Python code

6. Python 3.10+

What: Programming language **Why:**

- Easy to read and write
- Excellent AI/ML libraries (PyTorch, YOLO)
- MAVSDK has Python library
- FastAPI for microservices
- You already know it!

All your services are written in Python

7. Kafka Message Queue

What: Message delivery system (like postal service) **Why:**

- Services don't call each other directly
- Publishes to topics: `drone.telemetry.gps`
- Other services subscribe and read
- If service is down, messages wait in queue
- Can replay messages for debugging

Example:

```
# Service A publishes

producer.send('drone.telemetry.gps', {'lat': -1.29, 'lon': 36.82})



# Service B subscribes (different file, different process)

for message in consumer:

    print(message.value)  # Gets the GPS data
```

8. InfluxDB

What: Time-series database **Why:**

- Optimized for time-stamped data (telemetry)
- Stores: timestamp + latitude + longitude + altitude
- Very fast queries: "Show me GPS path from last 10 minutes"
- Grafana reads from InfluxDB

Regular database: Good for users, products, orders **InfluxDB:** Perfect for GPS readings every second

9. Grafana

What: Visualization dashboard **Why:**

- Beautiful graphs and maps
- Real-time updates (refreshes every second)
- Shows GPS path on world map
- Graphs for altitude, battery, speed
- Industry standard for monitoring

You will see your drone flying on a map in Grafana - just like flight tracking apps!

10. Docker & Docker Compose

What: Container platform **Why:**

- Kafka, InfluxDB, Grafana run in containers
- Easy to install (just `docker compose up`)
- Isolated from your system
- One command starts all services

Without Docker: Install Kafka (30 steps), Install InfluxDB (20 steps)... **With Docker:** One file, one command, everything works

11. AI/ML Libraries

PyTorch:

- Deep learning framework
- Used for Reinforcement Learning agent
- Trains neural network to fly drone

YOLOv8 (Ultralytics):

- Object detection AI
- Detects trees, animals, buildings in camera feed
- Very fast (30+ FPS)

spaCy / Transformers:

- Natural Language Processing
- Understands: "Go to forest and avoid lake"
- Extracts: Action=go, Location=forest, Avoid=lake

System Architecture (Complete Picture)

```
YOUR PROJECT DIRECTORY STRUCTURE:

~/fyp_project/

├── microservices/              ← All services here

│   ├── telemetry_collector/   ← Reads drone data

│   │   └── service.py

│   ├── flight_control/        ← Controls drone

│   │   └── service.py

│   ├── command_parser/        ← NLP AI

│   │   └── service.py

│   ├── object_detection/      ← YOLO AI

│   │   └── service.py

│   ├── rl_navigator/          ← RL AI

│   │   └── service.py

│   └── feedback_service/      ← Audio feedback

│       └── service.py

├── ai_models/                  ← Trained AI models

│   ├── yolo/

│   │   └── best.pt            ← Trained YOLO

│   ├── rl_agent/

│   │   └── agent.pth          ← Trained RL

│   └── nlp/

│       └── command_model/     ← NLP model

├── docker-compose.yml         ← Kafka, InfluxDB, Grafana

├── start_all.sh              ← Script to start everything

├── test_flight.py            ← Test drone control

└── README.md                 ← Instructions
```

```
RUNNING PROCESSES:

Terminal 1: PX4 SITL + Gazebo        (cd ~/PX4-Autopilot && make px4_sitl
gz_x500)

Terminal 2: Docker services          (docker compose up)

Terminal 3: Telemetry Collector     (python
microservices/telemetry_collector/service.py)

Terminal 4: Flight Control          (python
microservices/flight_control/service.py)

Terminal 5: Object Detection        (python
microservices/object_detection/service.py)

Terminal 6: RL Navigator           (python
microservices/rl_navigator/service.py)

Terminal 7: Command Parser         (python
microservices/command_parser/service.py)

Browser: Grafana                    (http://localhost:3000)


DATA FLOW:

You type command → Command Parser (NLP) → Kafka → Flight Control → PX4 →
Gazebo (drone moves)

Gazebo → PX4 → Telemetry Collector → Kafka → InfluxDB → Grafana (you see
movement)

Gazebo camera → Object Detection (YOLO) → Kafka → RL Navigator → Flight
Control (avoids obstacles)
```

PART 2: COMPLETE SETUP

Step 1: Optimize Ubuntu for Your 8GB RAM

Open terminal (Ctrl+Alt+T) and run:

```
# Reduce swap usage (saves RAM)

echo "vm.swappiness=10" | sudo tee -a /etc/sysctl.conf

sudo sysctl -p


# Disable unnecessary services
```

```
sudo systemctl disable bluetooth.service

sudo systemctl disable cups.service


# Check current RAM usage

free -h
```

Expected output:

```
              total        used        free      shared  buff/cache
available

Mem:          7.7Gi       2.5Gi       3.2Gi       200Mi       2.0Gi
4.8Gi
```

Tips for 8GB RAM:

- Close unnecessary applications when running simulation
- Use lightweight browser (Firefox instead of Chrome)
- Run Gazebo in "headless" mode when not needed (saves ~500MB)

Step 2: Update System

```
sudo apt update && sudo apt upgrade -y
```

This takes 10-15 minutes. Get coffee ☕

Step 3: Install ROS 2 Humble

Why ROS 2? Connects Gazebo, camera, sensors to your Python code.

```
# Set locale

sudo apt install locales

sudo locale-gen en_US en_US.UTF-8

sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8

export LANG=en_US.UTF-8


# Add ROS 2 repository

sudo apt install software-properties-common curl -y

sudo add-apt-repository universe -y
```

```
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key
-o /usr/share/keyrings/ros-archive-keyring.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/ros-archive-keyring.gpg]
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo
$UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list >
/dev/null
```

```
# Install ROS 2

sudo apt update

sudo apt install ros-humble-desktop -y
```

Installation takes 15-20 minutes.

Setup environment:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc

source ~/.bashrc
```

Test:

```
ros2 --version
```

Should show: `ros2 cli version 0.XX.X`

What just happened?

- ROS 2 libraries installed in `/opt/ros/humble/`
- When you run `source /opt/ros/humble/setup.bash`, it adds ROS 2 commands to your terminal
- Now you can use `ros2 topic list`, `ros2 run`, etc.

Step 4: Install Gazebo Garden

Why Gazebo? You'll SEE your drone flying in 3D.

```
# Add Gazebo repository

sudo apt-get install lsb-release wget gnupg -y

sudo wget https://packages.osrfoundation.org/gazebo.gpg -O
/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg]
http://packages.osrfoundation.org/gazebo/ubuntu-stable $(lsb_release -cs)
main" | sudo tee /etc/apt/sources.list.d/gazebo-stable.list > /dev/null


# Install Gazebo

sudo apt-get update

sudo apt-get install gz-garden -y
```

Takes 10-15 minutes.

Test:

```
gz sim shapes.sdf
```

You should see: 3D window with geometric shapes. Use mouse to rotate view!

What you're seeing:

- 3D physics simulation
- Real-time rendering
- This is where your drone will fly

Close Gazebo (Ctrl+C in terminal)

Step 5: Install PX4 SITL

Why PX4? This is the "brain" of the drone - the autopilot software.

```
# Clone PX4

cd ~

git clone https://github.com/PX4/PX4-Autopilot.git --recursive

cd PX4-Autopilot
```

Cloning takes 5-10 minutes (large repository).

Run setup script:

```
bash ./Tools/setup/ubuntu.sh
```

This installs:

- Compiler tools
- Python dependencies
- MAVLink libraries

Takes 20-30 minutes. Answer "Yes" to prompts.

IMPORTANT: Restart terminal after installation

Close terminal and open new one, then:

```
cd ~/PX4-Autopilot
```

```
make px4_sitl gz_x500
```

First build takes 15-20 minutes. Patience!

What's happening:

- PX4 firmware is being compiled
- It's configured for SITL (Software In The Loop) mode
- `gz_x500` means quadcopter drone model

Expected result:

1. Gazebo window opens showing a quadcopter
2. Terminal shows `pxh>` prompt (PX4 shell)
3. You see:

```
INFO [simulator] Waiting for simulator to accept connection
```

```
INFO [simulator] Simulator connected
```

Test basic command:

```
# In PX4 console (pxh> prompt), type:
```

```
commander takeoff
```

Drone should lift off in Gazebo! 🚁

```
commander land
```

Drone lands.

Press Ctrl+C to stop simulation.

What just happened?

- PX4 SITL is a "virtual flight controller"
- It runs on your computer but behaves like real Pixhawk hardware
- MAVLink protocol connects PX4 ↔ Gazebo ↔ Your Python code
- `commander takeoff` sent MAVLink command to PX4
- PX4 calculated motor speeds
- Gazebo simulated physics
- You saw realistic flight!

Step 6: Install QGroundControl

Why QGC? Professional ground control station - monitor and control drone with GUI.

```
cd ~/Downloads

wget https://d176tv9ibo4jno.cloudfront.net/latest/QGroundControl.AppImage

chmod +x QGroundControl.AppImage


# Install FUSE (required for AppImage)

sudo apt install libfuse2 -y


# Run QGC

./QGroundControl.AppImage
```

Test connection:

Terminal 1:

```
cd ~/PX4-Autopilot

make px4_sitl gz_x500
```

Terminal 2:

```
cd ~/Downloads

./QGroundControl.AppImage
```

QGC should show:

- Green "Connected" indicator
- GPS coordinates

- Battery 100%
- Altitude 0m

Try mission planning:

1. Click "Plan" tab
2. Click on map to add waypoints
3. Click "Upload" to drone
4. Switch to "Fly" tab
5. Click "Start Mission"

Drone follows waypoints in Gazebo!

What's happening:

- QGC sends MAVLink waypoints to PX4
- PX4 calculates flight path
- Gazebo simulates the flight
- You see it all happen!

Step 7: Python Environment

```
# Create virtual environment

cd ~

python3 -m venv fyp_env

source fyp_env/bin/activate


# You'll see (fyp_env) in prompt


# Upgrade pip

pip install --upgrade pip


# Install all packages

pip install pymavlink mavsdk asyncio

pip install fastapi uvicorn

pip install kafka-python

pip install influxdb-client

pip install opencv-python opencv-contrib-python

pip install ultralytics
```

```
pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu

pip install stable-baselines3 gymnasium

pip install spacy transformers

pip install pyttsx3 SpeechRecognition

pip install numpy pandas matplotlib requests pyyaml

pip install pytest pytest-asyncio
```

Takes 10-15 minutes.

Download NLP model:

```
python -m spacy download en_core_web_sm
```

Test installations:

```
python3 << 'EOF'

import mavsdk

import fastapi

import kafka

import cv2

import torch

import spacy

print("✓ All packages installed successfully!")

EOF
```

What are these packages?

- `mavsdk`: Talk to drone (send commands, get telemetry)
- `fastapi`: Build REST APIs for microservices
- `kafka-python`: Connect to Kafka message queue
- `influxdb-client`: Write telemetry to database
- `opencv-python`: Process camera images
- `ultralytics`: YOLOv8 object detection
- `torch`: Deep learning (RL agent)
- `spacy`: Natural language processing

Step 8: Docker & Docker Compose

Why Docker? One command to start Kafka, InfluxDB, Grafana.

```
# Remove old versions

sudo apt-get remove docker docker-engine docker.io containerd runc



# Install dependencies

sudo apt-get update

sudo apt-get install ca-certificates curl gnupg lsb-release -y



# Add Docker repository

sudo mkdir -p /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor
-o /etc/apt/keyrings/docker.gpg



echo \

  "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \

  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list >
/dev/null



# Install Docker

sudo apt-get update

sudo apt-get install docker-ce docker-ce-cli containerd.io docker-compose-
plugin -y



# Add your user to docker group (IMPORTANT!)

sudo usermod -aG docker $USER
```

LOGOUT AND LOGIN AGAIN! This is crucial for Docker permissions.

After logging back in, test:

```
docker --version

docker compose version
```

```
docker run hello-world
```

Should see: "Hello from Docker!"

What is Docker?

- Runs applications in "containers" (isolated environments)
- Like lightweight virtual machines
- `docker-compose.yml` file describes all containers
- One command starts everything

Step 9: Create Project Structure

```
mkdir -p ~/fyp_project

cd ~/fyp_project


# Create all directories

mkdir -p
microservices/{telemetry_collector,flight_control,command_parser,object_detec
tion,rl_navigator,feedback_service}

mkdir -p ai_models/{nlp,yolo,rl_agent}

mkdir -p config data logs tests

mkdir -p grafana_dashboards
```

Your directory structure:

```
~/fyp_project/

├── microservices/

│   ├── telemetry_collector/

│   ├── flight_control/

│   ├── command_parser/

│   ├── object_detection/

│   ├── rl_navigator/

│   └── feedback_service/

├── ai_models/

│   ├── nlp/

│   ├── yolo/
```

```
│   └── rl_agent/
├── config/
├── data/
├── logs/
├── tests/
└── grafana_dashboards/
```

What goes where:

- `microservices/`: All Python services
- `ai_models/`: Trained AI model files (.pt, .pth)
- `config/`: Configuration files
- `data/`: Training data for AI
- `logs/`: Log files
- `tests/`: Test scripts
- `grafana_dashboards/`: Dashboard JSON files

Step 10: Docker Compose Configuration

Create the file:

```
cd ~/fyp_project
nano docker-compose.yml
```

Paste this (READ THE COMMENTS - they explain everything):

```
version: '3.8'


services:
  # ============================================
  # ZOOKEEPER - Manages Kafka cluster
  # Kafka needs this to coordinate
  # ============================================
  zookeeper:
    image: confluentinc/cp-zookeeper:latest
    container_name: zookeeper
    environment:
```

```
    ZOOKEEPER_CLIENT_PORT: 2181
    ZOOKEEPER_TICK_TIME: 2000
  ports:
    - "2181:2181"
  networks:
    - fyp_network
  # Limit memory for your 8GB RAM
  mem_limit: 512m

# ============================================
# KAFKA - Message Queue (Post Office)
# Services publish/subscribe to topics here
# ============================================
kafka:
  image: confluentinc/cp-kafka:latest
  container_name: kafka
  depends_on:
    - zookeeper
  ports:
    - "9092:9092"
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
    # Memory optimization
    KAFKA_HEAP_OPTS: "-Xmx512M -Xms512M"
  networks:
    - fyp_network
  mem_limit: 1g
```

```yaml
# ============================================
# INFLUXDB - Time-Series Database
# Stores: timestamp + GPS + altitude + battery
# Optimized for telemetry data
# ============================================
influxdb:
  image: influxdb:2.7
  container_name: influxdb
  ports:
    - "8086:8086"
  environment:
    # Initial setup credentials
    DOCKER_INFLUXDB_INIT_MODE: setup
    DOCKER_INFLUXDB_INIT_USERNAME: admin
    DOCKER_INFLUXDB_INIT_PASSWORD: adminpass123
    DOCKER_INFLUXDB_INIT_ORG: fyp_org
    DOCKER_INFLUXDB_INIT_BUCKET: drone_telemetry
    DOCKER_INFLUXDB_INIT_ADMIN_TOKEN: my-super-secret-token-12345
  volumes:
    # Data persists even if container restarts
    - influxdb_data:/var/lib/influxdb2
  networks:
    - fyp_network
  mem_limit: 1g

# ============================================
# GRAFANA - Visualization Dashboard
# You'll see: GPS maps, graphs, gauges
# ============================================
grafana:
```

```
  image: grafana/grafana:latest
  container_name: grafana
  ports:
    - "3000:3000"
  environment:
    # Login credentials
    GF_SECURITY_ADMIN_USER: admin
    GF_SECURITY_ADMIN_PASSWORD: admin
    # Install plugins
    GF_INSTALL_PLUGINS: grafana-worldmap-panel,grafana-clock-panel
  volumes:
    - grafana_data:/var/lib/grafana
    - ./grafana_dashboards:/etc/grafana/provisioning/dashboards
  depends_on:
    - influxdb
  networks:
    - fyp_network
  mem_limit: 512m

# ============================================
# POSTGRESQL - Relational Database
# For logs, user data, mission history
# ============================================
postgres:
  image: postgres:15
  container_name: postgres
  environment:
    POSTGRES_DB: fyp_db
    POSTGRES_USER: fyp_user
    POSTGRES_PASSWORD: fyp_pass
  ports:
    - "5432:5432"
```

```
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - fyp_network
    mem_limit: 512m

# ============================================
# VOLUMES - Persistent data storage
# Data survives container restart/removal
# ============================================
volumes:
  influxdb_data:
  grafana_data:
  postgres_data:

# ============================================
# NETWORK - All containers communicate here
# ============================================
networks:
  fyp_network:
    driver: bridge
```

Save: Ctrl+O, Enter, Ctrl+X

Explanation of each service:

Zookeeper:

- Manages Kafka cluster
- Think of it as Kafka's coordinator
- You don't interact with it directly

Kafka:

- Message queue system
- Services publish messages to "topics"
- Other services subscribe and read messages

- Topics you'll use:
- `drone.telemetry.gps` - GPS data
- `drone.commands.flight` - Flight commands
- `drone.detections.objects` - YOLO detections
- `drone.navigation.decisions` - RL decisions

InfluxDB:

- Time-series database
- Perfect for telemetry (data with timestamps)
- Stores billions of data points efficiently
- Fast queries: "Show GPS path from 14:30 to 14:45"

Grafana:

- Visualization tool
- Connects to InfluxDB
- Displays data as:
- Line charts (altitude over time)
- Maps (GPS path)
- Gauges (battery percentage)
- Tables (log entries)

PostgreSQL:

- Regular relational database
- Stores mission logs, user data, system configuration
- When you run a mission, it's saved here

Start all services:

```
cd ~/fyp_project
```

```
docker compose up -d
```

First time takes 5-10 minutes (downloads images).

`-d` means "detached mode" (runs in background)

Check status:

```
docker compose ps
```

Should see:

```
NAME          STATUS
kafka         Up
zookeeper     Up
```

```
influxdb     Up

grafana      Up

postgres     Up
```

All should be "Up"!

Access services:

- InfluxDB: http://localhost:8086 (admin / adminpass123)
- Grafana: http://localhost:3000 (admin / admin)
- Kafka: localhost:9092 (code connects here)
- PostgreSQL: localhost:5432 (code connects here)

Step 11: Create Kafka Topics

Kafka topics are like "channels" - services publish/subscribe to specific channels.

```
# Enter Kafka container

docker exec -it kafka bash



# Create all topics

kafka-topics --create --topic drone.telemetry.gps --bootstrap-server
localhost:9092 --partitions 1 --replication-factor 1



kafka-topics --create --topic drone.telemetry.battery --bootstrap-server
localhost:9092 --partitions 1 --replication-factor 1



kafka-topics --create --topic drone.telemetry.attitude --bootstrap-server
localhost:9092 --partitions 1 --replication-factor 1



kafka-topics --create --topic drone.telemetry.velocity --bootstrap-server
localhost:9092 --partitions 1 --replication-factor 1



kafka-topics --create --topic drone.commands.flight --bootstrap-server
localhost:9092 --partitions 1 --replication-factor 1



kafka-topics --create --topic drone.commands.parsed --bootstrap-server
localhost:9092 --partitions 1 --replication-factor 1
```

```
kafka-topics --create --topic drone.detections.objects --bootstrap-server
localhost:9092 --partitions 1 --replication-factor 1


kafka-topics --create --topic drone.navigation.decisions --bootstrap-server
localhost:9092 --partitions 1 --replication-factor 1


kafka-topics --create --topic drone.feedback.messages --bootstrap-server
localhost:9092 --partitions 1 --replication-factor 1


kafka-topics --create --topic drone.alerts.critical --bootstrap-server
localhost:9092 --partitions 1 --replication-factor 1


kafka-topics --create --topic drone.status.system --bootstrap-server
localhost:9092 --partitions 1 --replication-factor 1


# List topics to verify

kafka-topics --list --bootstrap-server localhost:9092


# Exit container

exit
```

What are these topics for?

Topic	Publisher	Subscriber	Data
`drone.telemetry.gps`	Telemetry Collector	InfluxDB writer, RL Navigator	GPS coordinates
`drone.telemetry.battery`	Telemetry Collector	Dashboard, Alert system	Battery %
`drone.commands.flight`	Command Parser	Flight Control	Takeoff, land, goto
`drone.detections.objects`	Object Detection	RL Navigator	Detected obstacles
`drone.navigation.decisions`	RL Navigator	Flight Control	Avoidance commands

Step 12: Setup Grafana Data Source

Open browser: http://localhost:3000

Login: admin / admin (skip password change for now)

Add InfluxDB data source:

1. Click ⚙️ (Settings) → Data Sources
2. Click "Add data source"
3. Select "InfluxDB"
4. Configure: `Name: InfluxDB-DroneQuery Language: FluxURL: http://influxdb:8086Organization: fyp_orgToken: my-super-secret-token-12345Default Bucket: drone_telemetry`
5.
6. Click "Save & Test"
7. Should see green checkmark: "datasource is working"

What just happened?

- Grafana can now query InfluxDB
- When you create graphs, they'll pull data from InfluxDB
- Real-time updates (queries every second)

🎉 **SETUP COMPLETE! All infrastructure is ready.**

Summary of what's running:

- ✅ Ubuntu 22.04
- ✅ ROS 2 Humble
- ✅ Gazebo Garden
- ✅ PX4 SITL
- ✅ QGroundControl
- ✅ Python environment
- ✅ Docker containers (Kafka, InfluxDB, Grafana, PostgreSQL)
- ✅ Kafka topics created
- ✅ Grafana connected to InfluxDB

Next: Build the microservices!

PART 3: PROJECT STRUCTURE IN DETAIL

Complete Directory Map

```
~/fyp_project/                          ← Main project folder
│
├── docker-compose.yml                  ← Starts Kafka, InfluxDB, Grafana
│
├── microservices/                      ← All Python services
```

```
│   │
│   ├── telemetry_collector/          ← Service 1
│   │   ├── service.py                ← Main code (reads drone data)
│   │   ├── config.py                 ← Configuration (ports, addresses)
│   │   └── requirements.txt          ← Python packages needed
│   │
│   ├── flight_control/               ← Service 2
│   │   ├── service.py                ← Controls drone (takeoff, land, goto)
│   │   ├── config.py
│   │   └── requirements.txt
│   │
│   ├── command_parser/               ← Service 3 (NLP AI)
│   │   ├── service.py                ← Parses user commands
│   │   ├── nlp_model.py              ← NLP logic
│   │   ├── config.py
│   │   └── requirements.txt
│   │
│   ├── object_detection/             ← Service 4 (YOLO AI)
│   │   ├── service.py                ← Runs YOLO on camera feed
│   │   ├── yolo_detector.py          ← YOLO logic
│   │   ├── config.py
│   │   └── requirements.txt
│   │
│   ├── rl_navigator/                 ← Service 5 (RL AI)
│   │   ├── service.py                ← Makes navigation decisions
│   │   ├── rl_agent.py               ← RL logic
│   │   ├── train_agent.py            ← Training script
│   │   ├── config.py
│   │   └── requirements.txt
│   │
│   └── feedback_service/             ← Service 6
│       ├── service.py                ← Provides audio/text feedback
```

```
│       ├── config.py
│       └── requirements.txt
│
├── ai_models/                          ← Trained AI models
│   ├── yolo/
│   │   └── best.pt                    ← Trained YOLO weights
│   ├── rl_agent/
│   │   └── navigation_agent.pth       ← Trained RL model
│   └── nlp/
│       └── command_parser/            ← NLP model files
│
├── config/                             ← Configuration files
│   ├── kafka_topics.txt               ← List of topics
│   ├── system_config.yaml             ← System settings
│   └── drone_params.yaml              ← Drone parameters
│
├── data/                               ← Training data
│   ├── training_images/               ← Images for YOLO training
│   ├── rl_episodes/                   ← RL training episodes
│   └── commands_dataset.csv           ← NLP training data
│
├── logs/                               ← Log files
│   ├── telemetry_YYYYMMDD.log
│   ├── flight_control_YYYYMMDD.log
│   └── system_YYYYMMDD.log
│
├── tests/                              ← Test scripts
│   ├── test_kafka.py                  ← Test Kafka connection
│   ├── test_influxdb.py               ← Test InfluxDB
│   ├── test_drone_connection.py       ← Test PX4 connection
│   └── test_full_system.py            ← Integration test
│
```

```
├── grafana_dashboards/                 ← Grafana dashboards
│   ├── main_dashboard.json            ← Main telemetry dashboard
│   └── system_health.json             ← System monitoring
│
├── scripts/                            ← Utility scripts
│   ├── start_all.sh                   ← Starts all services
│   ├── stop_all.sh                    ← Stops all services
│   ├── create_topics.sh               ← Creates Kafka topics
│   └── reset_database.sh              ← Clears InfluxDB
│
├── notebooks/                          ← Jupyter notebooks
│   ├── train_yolo.ipynb               ← Train YOLO
│   ├── train_rl.ipynb                 ← Train RL agent
│   └── analyze_telemetry.ipynb        ← Analyze data
│
└── README.md                           ← Project documentation
```

How Files Connect (Data Flow)

Example: User Command to Drone Flight

```
1. You type in terminal: "Go to forest area"
   ↓
2. command_parser/service.py receives input
   ↓
3. command_parser/nlp_model.py processes with NLP AI
   ↓
4. Converts to: {"type": "goto", "lat": -1.2921, "lon": 36.8219}
   ↓
5. Publishes to Kafka topic: drone.commands.flight
   ↓
6. flight_control/service.py subscribes to that topic
   ↓
```

```
7. flight_control/service.py receives the message
   ↓
8. Sends MAVLink command to PX4 SITL
   ↓
9. PX4 flies drone in Gazebo
   ↓
10. Drone position changes
    ↓
11. telemetry_collector/service.py reads new position
    ↓
12. Publishes to: drone.telemetry.gps
    ↓
13. Also writes to InfluxDB
    ↓
14. Grafana queries InfluxDB
    ↓
15. You see drone moving on map! 🎉
```

How to Open in IDE (VS Code)

Install VS Code:

```
sudo snap install code --classic
```

Open project:

```
cd ~/fyp_project
code .
```

VS Code will show directory tree on left:

```
EXPLORER
  fyp_project/
    ▼ microservices/
```

```
      ▼ telemetry_collector/
        service.py
        config.py
      ▼ flight_control/
        service.py
      ...
    ▼ ai_models/
    ▼ config/
    docker-compose.yml
```

Install VS Code extensions:

1. Click Extensions (Ctrl+Shift+X)
2. Search and install:
   - Python
   - Pylance
   - Docker
   - YAML

Now you can:

- Click any file to edit
- Right-click folder → New File
- Integrated terminal at bottom (Ctrl+`)
- Run Python files directly

PART 4: BUILDING THE SYSTEM - COMPLETE CODE

Service 1: Telemetry Collector

Purpose: Reads telemetry from PX4, publishes to Kafka, writes to InfluxDB.

File: ~/fyp_project/microservices/telemetry_collector/service.py

```
"""
TELEMETRY COLLECTOR SERVICE
===========================
What it does:
```

```
1. Connects to PX4 SITL via MAVLink

2. Continuously reads: GPS, battery, altitude, heading, speed

3. Publishes to Kafka topics (other services subscribe)

4. Writes to InfluxDB (Grafana reads from here)


How it works:

- Uses MAVSDK library to connect to drone

- Async functions run concurrently (collect GPS + battery + attitude at same
time)

- Every 1 second, new data is collected

"""


import asyncio

import json

import time

from mavsdk import System

from kafka import KafkaProducer

from influxdb_client import InfluxDBClient, Point, WritePrecision

from influxdb_client.client.write_api import SYNCHRONOUS


# ============================================

# CONFIGURATION

# ============================================

DRONE_ADDRESS = "udp://:14540"  # PX4 SITL default port

KAFKA_BROKER = "localhost:9092"

INFLUX_URL = "http://localhost:8086"

INFLUX_TOKEN = "my-super-secret-token-12345"

INFLUX_ORG = "fyp_org"

INFLUX_BUCKET = "drone_telemetry"

DRONE_ID = "drone_001"


class TelemetryCollector:
```

```
def __init__(self):
    # Initialize connections (will be established in connect())
    self.drone = System()
    self.kafka_producer = None
    self.influx_client = None
    self.influx_write_api = None

async def connect(self):
    """
    Establish all connections:
    1. Connect to PX4 SITL
    2. Connect to Kafka
    3. Connect to InfluxDB
    """
    print("=" * 50)
    print("TELEMETRY COLLECTOR SERVICE STARTING")
    print("=" * 50)

    # Connect to drone
    print(f"Connecting to drone at {DRONE_ADDRESS}...")
    await self.drone.connect(system_address=DRONE_ADDRESS)

    # Wait for connection
    print("Waiting for drone to connect...")
    async for state in self.drone.core.connection_state():
        if state.is_connected:
            print("✓ Drone connected!")
            break

    # Setup Kafka producer
    print("Connecting to Kafka...")
```

```
        self.kafka_producer = KafkaProducer(
            bootstrap_servers=[KAFKA_BROKER],
            value_serializer=lambda v: json.dumps(v).encode('utf-8')
        )
        print("✓ Kafka connected!")

        # Setup InfluxDB client
        print("Connecting to InfluxDB...")
        self.influx_client = InfluxDBClient(
            url=INFLUX_URL,
            token=INFLUX_TOKEN,
            org=INFLUX_ORG
        )
        self.influx_write_api =
self.influx_client.write_api(write_options=SYNCHRONOUS)
        print("✓ InfluxDB connected!")
        print("=" * 50)
        print()

    async def collect_gps(self):
        """
        Collect GPS position telemetry
        Runs continuously - async for loop never ends
        """
        print("[GPS] Starting GPS telemetry collection...")

        async for position in self.drone.telemetry.position():
            # Create telemetry message
            telemetry = {
                'drone_id': DRONE_ID,
                'type': 'gps',
                'latitude': position.latitude_deg,
```

```
                'longitude': position.longitude_deg,
                'altitude_msl': position.absolute_altitude_m,  # Mean Sea
Level
                'altitude_rel': position.relative_altitude_m,  # Relative to
takeoff
                'timestamp': time.time()
            }

            # Publish to Kafka
            # Other services can subscribe to 'drone.telemetry.gps' to get
this data
            self.kafka_producer.send('drone.telemetry.gps', telemetry)

            # Write to InfluxDB
            # Point = one data point with timestamp
            # .tag() = indexable metadata (for filtering queries)
            # .field() = actual data values
            point = Point("gps_position") \
                .tag("drone_id", DRONE_ID) \
                .field("latitude", position.latitude_deg) \
                .field("longitude", position.longitude_deg) \
                .field("altitude_rel", position.relative_altitude_m) \
                .time(int(time.time() * 1e9), WritePrecision.NS)  #
Nanosecond precision

            self.influx_write_api.write(
                bucket=INFLUX_BUCKET,
                org=INFLUX_ORG,
                record=point
            )

            # Print to console (helps with debugging)
            print(f"[GPS] Lat: {position.latitude_deg:.6f}, "
```

```
                f"Lon: {position.longitude_deg:.6f}, "
                f"Alt: {position.relative_altitude_m:.2f}m")

        await asyncio.sleep(1)  # Collect every 1 second

    async def collect_battery(self):
        """
        Collect battery telemetry
        Important for safety - low battery triggers return-to-home
        """
        print("[BATTERY] Starting battery telemetry collection...")

        async for battery in self.drone.telemetry.battery():
            telemetry = {
                'drone_id': DRONE_ID,
                'type': 'battery',
                'voltage': battery.voltage_v,
                'remaining_percent': battery.remaining_percent,
                'timestamp': time.time()
            }

            self.kafka_producer.send('drone.telemetry.battery', telemetry)

            point = Point("battery_status") \
                .tag("drone_id", DRONE_ID) \
                .field("voltage", battery.voltage_v) \
                .field("remaining_percent", battery.remaining_percent) \
                .time(int(time.time() * 1e9), WritePrecision.NS)

            self.influx_write_api.write(
                bucket=INFLUX_BUCKET,
```

```
            org=INFLUX_ORG,
            record=point
        )

        # Color-code battery level for visibility
        if battery.remaining_percent < 20:
            status = "⚠️  LOW"
        elif battery.remaining_percent < 50:
            status = "⚡"
        else:
            status = "✓"

        print(f"[BATTERY] {status} {battery.remaining_percent:.1f}% "
              f"({battery.voltage_v:.2f}V)")

        await asyncio.sleep(2)  # Every 2 seconds

    async def collect_attitude(self):
        """
        Collect attitude (orientation) telemetry
        Roll, Pitch, Yaw = rotation around X, Y, Z axes
        Yaw = heading/compass direction
        """
        print("[ATTITUDE] Starting attitude telemetry collection...")

        async for attitude in self.drone.telemetry.attitude_euler():
            telemetry = {
                'drone_id': DRONE_ID,
                'type': 'attitude',
                'roll': attitude.roll_deg,    # Tilt left/right
                'pitch': attitude.pitch_deg,  # Tilt forward/back
```

```
                'yaw': attitude.yaw_deg,      # Compass heading (0=North,
90=East)
                'timestamp': time.time()
            }

            self.kafka_producer.send('drone.telemetry.attitude', telemetry)

            point = Point("attitude") \
                .tag("drone_id", DRONE_ID) \
                .field("roll", attitude.roll_deg) \
                .field("pitch", attitude.pitch_deg) \
                .field("yaw", attitude.yaw_deg) \
                .time(int(time.time() * 1e9), WritePrecision.NS)

            self.influx_write_api.write(
                bucket=INFLUX_BUCKET,
                org=INFLUX_ORG,
                record=point
            )

            # Determine cardinal direction from yaw
            directions = ['N', 'NE', 'E', 'SE', 'S', 'SW', 'W', 'NW']
            dir_idx = int((attitude.yaw_deg + 22.5) / 45) % 8
            direction = directions[dir_idx]

            print(f"[ATTITUDE] Heading: {attitude.yaw_deg:.1f}°
({direction})")

            await asyncio.sleep(1)

    async def collect_velocity(self):
        """
```

```
Collect velocity telemetry
NED = North-East-Down coordinate system
"""
print("[VELOCITY] Starting velocity telemetry collection...")

async for velocity in self.drone.telemetry.velocity_ned():
    # Calculate horizontal speed (Pythagorean theorem)
    speed = (velocity.north_m_s**2 + velocity.east_m_s**2)**0.5

    telemetry = {
        'drone_id': DRONE_ID,
        'type': 'velocity',
        'north': velocity.north_m_s,
        'east': velocity.east_m_s,
        'down': velocity.down_m_s,
        'speed': speed,
        'timestamp': time.time()
    }

    self.kafka_producer.send('drone.telemetry.velocity', telemetry)

    point = Point("velocity") \
        .tag("drone_id", DRONE_ID) \
        .field("speed", speed) \
        .field("north", velocity.north_m_s) \
        .field("east", velocity.east_m_s) \
        .field("down", velocity.down_m_s) \
        .time(int(time.time() * 1e9), WritePrecision.NS)

    self.influx_write_api.write(
        bucket=INFLUX_BUCKET,
```

```
                org=INFLUX_ORG,
                record=point
            )

            print(f"[VELOCITY] Speed: {speed:.2f} m/s "
                  f"(N:{velocity.north_m_s:.1f} E:{velocity.east_m_s:.1f} D:
{velocity.down_m_s:.1f})")

            await asyncio.sleep(1)

    async def run(self):
        """
        Main run loop
        Runs all collectors concurrently using asyncio.gather()
        """
        await self.connect()

        print("Starting all telemetry collectors...")
        print()

        # Run all collectors at the same time
        # If any crashes, they all stop (can change this behavior if needed)
        await asyncio.gather(
            self.collect_gps(),
            self.collect_battery(),
            self.collect_attitude(),
            self.collect_velocity()
        )

    def shutdown(self):
        """Clean shutdown - close all connections"""
        print("\nShutting down telemetry collector...")
```

```
        if self.kafka_producer:
            self.kafka_producer.close()
        if self.influx_client:
            self.influx_client.close()
        print("✓ Shutdown complete")

# ============================================
# MAIN ENTRY POINT
# ============================================
async def main():
    """
    Main function
    Creates collector and runs it
    Handles Ctrl+C gracefully
    """
    collector = TelemetryCollector()

    try:
        await collector.run()
    except KeyboardInterrupt:
        print("\n\n⚠️  Keyboard interrupt received...")
        collector.shutdown()
    except Exception as e:
        print(f"\n\n❌ Error: {e}")
        collector.shutdown()

if __name__ == "__main__":
    """
    Python starts here
    asyncio.run() is the entry point for async code
    """
```

```
    asyncio.run(main())
```

How to run:

Terminal 1: Start PX4

```
cd ~/PX4-Autopilot
make px4_sitl gz_x500
```

Terminal 2: Start Docker services

```
cd ~/fyp_project
docker compose up
```

Terminal 3: Activate env and run

```
source ~/fyp_env/bin/activate
cd ~/fyp_project/microservices/telemetry_collector
python service.py
```

Expected output:

```
==================================================
TELEMETRY COLLECTOR SERVICE STARTING
==================================================
Connecting to drone at udp://:14540...
Waiting for drone to connect...
✓ Drone connected!
Connecting to Kafka...
✓ Kafka connected!
Connecting to InfluxDB...
✓ InfluxDB connected!
==================================================

Starting all telemetry collectors...
```

```
[GPS] Starting GPS telemetry collection...

[BATTERY] Starting battery telemetry collection...

[ATTITUDE] Starting attitude telemetry collection...

[VELOCITY] Starting velocity telemetry collection...

[GPS] Lat: 47.397742, Lon: 8.545594, Alt: 0.00m

[BATTERY] ✓ 100.0% (12.60V)

[ATTITUDE] Heading: 0.0° (N)

[VELOCITY] Speed: 0.00 m/s (N:0.0 E:0.0 D:0.0)

...
```

Verify data in Grafana:

1. Open http://localhost:3000
2. Login: admin / admin
3. Click + → Dashboard → Add visualization
4. Select "InfluxDB-Drone"
5. Query editor, paste:

```
from(bucket: "drone_telemetry")

  |> range(start: -5m)

  |> filter(fn: (r) => r._measurement == "gps_position")

  |> filter(fn: (r) => r._field == "altitude_rel")
```

1. Click "Run query"
2. You should see altitude data!

Service 2: Flight Control

Purpose: Receives commands via Kafka and controls the drone.

File: ~/fyp_project/microservices/flight_control/service.py

```
"""

FLIGHT CONTROL SERVICE

======================

What it does:

1. Subscribes to Kafka topic: drone.commands.flight
```

```
2. Receives commands like: {"type": "takeoff"} or {"type": "goto", "lat": -
1.29, "lon": 36.82}

3. Sends MAVLink commands to PX4

4. Drone executes in Gazebo

5. Publishes status back to Kafka


Commands supported:

- arm: Arm motors

- takeoff: Take off to default altitude

- land: Land at current position

- goto: Fly to GPS coordinates

- rtl: Return to launch position

- hold: Hold current position

"""


import asyncio

import json

import time

from mavsdk import System

from kafka import KafkaConsumer, KafkaProducer


# ============================================

# CONFIGURATION

# ============================================

DRONE_ADDRESS = "udp://:14540"

KAFKA_BROKER = "localhost:9092"

DRONE_ID = "drone_001"


class FlightController:

    def __init__(self):

        self.drone = System()

        self.consumer = None
```

```
        self.producer = None
        self.is_armed = False
        self.is_flying = False

    async def connect(self):
        """Connect to drone and Kafka"""
        print("=" * 50)
        print("FLIGHT CONTROL SERVICE STARTING")
        print("=" * 50)

        # Connect to drone
        print(f"Connecting to drone at {DRONE_ADDRESS}...")
        await self.drone.connect(system_address=DRONE_ADDRESS)

        async for state in self.drone.core.connection_state():
            if state.is_connected:
                print("✓ Drone connected!")
                break

        # Setup Kafka consumer (receives commands)
        print("Setting up Kafka consumer...")
        self.consumer = KafkaConsumer(
            'drone.commands.flight',  # Subscribe to this topic
            bootstrap_servers=[KAFKA_BROKER],
            auto_offset_reset='latest',  # Only new messages (not historical)
            value_deserializer=lambda m: json.loads(m.decode('utf-8'))
        )
        print("✓ Kafka consumer ready!")

        # Setup Kafka producer (publishes status)
        print("Setting up Kafka producer...")
```

```
        self.producer = KafkaProducer(
            bootstrap_servers=[KAFKA_BROKER],
            value_serializer=lambda v: json.dumps(v).encode('utf-8')
        )
        print("✓ Kafka producer ready!")
        print("=" * 50)
        print("\nWaiting for commands on 'drone.commands.flight'...")
        print()

    async def execute_command(self, command):
        """
        Execute flight command

        Args:
            command (dict): Command from Kafka
                Examples:
                {"type": "takeoff"}
                {"type": "goto", "latitude": -1.29, "longitude": 36.82,
"altitude": 50}
                {"type": "land"}
        """
        cmd_type = command.get('type')

        print(f"\n{'='*50}")
        print(f"RECEIVED COMMAND: {cmd_type}")
        print(f"Full command: {command}")
        print(f"{'='*50}")

        try:
            if cmd_type == 'arm':
                print("[ARM] Arming motors...")
                await self.drone.action.arm()
```

```
                self.is_armed = True
                self.publish_status("arm", "success", "Motors armed")
                print("✓ Motors armed!")

            elif cmd_type == 'takeoff':
                if not self.is_armed:
                    print("[ARM] Arming first...")
                    await self.drone.action.arm()
                    await asyncio.sleep(1)

                altitude = command.get('altitude', 10)  # Default 10m
                print(f"[TAKEOFF] Taking off to {altitude}m...")
                await self.drone.action.set_takeoff_altitude(altitude)
                await self.drone.action.takeoff()
                self.is_flying = True
                self.is_armed = True
                self.publish_status("takeoff", "success", f"Took off to
{altitude}m")
                print(f"✓ Taking off to {altitude}m!")

            elif cmd_type == 'land':
                print("[LAND] Landing...")
                await self.drone.action.land()
                self.is_flying = False
                self.publish_status("land", "success", "Landing")
                print("✓ Landing initiated!")

            elif cmd_type == 'goto':
                lat = command.get('latitude')
                lon = command.get('longitude')
                alt = command.get('altitude', 50)  # Default 50m
```

```
                if lat is None or lon is None:
                    raise ValueError("latitude and longitude required for
goto")

                print(f"[GOTO] Flying to:")
                print(f"  Latitude:  {lat}")
                print(f"  Longitude: {lon}")
                print(f"  Altitude:  {alt}m")

                # goto_location(latitude, longitude, altitude, yaw)
                await self.drone.action.goto_location(lat, lon, alt, 0)
                self.publish_status("goto", "success", f"Going to {lat},
{lon}")
                print("✓ Goto command sent!")

            elif cmd_type == 'rtl':  # Return To Launch
                print("[RTL] Returning to launch position...")
                await self.drone.action.return_to_launch()
                self.publish_status("rtl", "success", "Returning home")
                print("✓ RTL activated!")

            elif cmd_type == 'hold':
                print("[HOLD] Holding position...")
                await self.drone.action.hold()
                self.publish_status("hold", "success", "Holding position")
                print("✓ Holding position!")

            else:
                error_msg = f"Unknown command type: {cmd_type}"
                print(f"❌ {error_msg}")
                self.publish_status(cmd_type, "error", error_msg)
```

```
        except Exception as e:
            error_msg = str(e)
            print(f"❌ Error executing {cmd_type}: {error_msg}")
            self.publish_status(cmd_type, "error", error_msg)

        print(f"{'='*50}\n")

    def publish_status(self, command, status, message):
        """
        Publish command execution status to Kafka
        Other services can monitor this
        """
        status_msg = {
            'drone_id': DRONE_ID,
            'command': command,
            'status': status,
            'message': message,
            'is_armed': self.is_armed,
            'is_flying': self.is_flying,
            'timestamp': time.time()
        }

        self.producer.send('drone.status.flight', status_msg)

    async def run(self):
        """
        Main run loop
        Continuously listen for commands from Kafka
        """
        await self.connect()
```

```
        # Create async task for command listening
        # This allows the drone connection to stay alive
        loop = asyncio.get_event_loop()

        while True:
            # Poll Kafka for new messages
            # timeout_ms=1000 means check every second
            messages = self.consumer.poll(timeout_ms=1000)

            if messages:
                for topic_partition, records in messages.items():
                    for record in records:
                        command = record.value
                        # Execute command asynchronously
                        await self.execute_command(command)

            await asyncio.sleep(0.1)  # Small delay to prevent CPU overuse

    def shutdown(self):
        """Clean shutdown"""
        print("\nShutting down flight controller...")
        if self.consumer:
            self.consumer.close()
        if self.producer:
            self.producer.close()
        print("✓ Shutdown complete")

# ==============================================
# MAIN ENTRY POINT
# ==============================================
async def main():
```

```
    controller = FlightController()

    try:
        await controller.run()
    except KeyboardInterrupt:
        print("\n\n⚠️  Keyboard interrupt received...")
        controller.shutdown()
    except Exception as e:
        print(f"\n\n❌ Error: {e}")
        controller.shutdown()

if __name__ == "__main__":
    asyncio.run(main())
```

Test the flight control:

Terminal 4: Start Flight Control

```
source ~/fyp_env/bin/activate
cd ~/fyp_project/microservices/flight_control
python service.py
```

Terminal 5: Send test command

```
source ~/fyp_env/bin/activate
cd ~/fyp_project
python3 << 'EOF'
import json
from kafka import KafkaProducer
import time

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
```

```
)

# Send takeoff command
command = {'type': 'takeoff', 'altitude': 20}
producer.send('drone.commands.flight', command)
print(f"Sent command: {command}")

time.sleep(2)

# Send goto command
command = {
    'type': 'goto',
    'latitude': 47.398,
    'longitude': 8.546,
    'altitude': 50
}
producer.send('drone.commands.flight', command)
print(f"Sent command: {command}")

producer.close()
EOF
```

Watch Terminal 4 - should see:

```
==================================================
RECEIVED COMMAND: takeoff
Full command: {'type': 'takeoff', 'altitude': 20}
==================================================
[ARM] Arming first...
[TAKEOFF] Taking off to 20m...
✓ Taking off to 20m!
==================================================
```

```
==================================================

RECEIVED COMMAND: goto

Full command: {'type': 'goto', 'latitude': 47.398, 'longitude': 8.546,
'altitude': 50}

==================================================

[GOTO] Flying to:

  Latitude:  47.398

  Longitude: 8.546

  Altitude:  50m

✓ Goto command sent!

==================================================
```

Watch Gazebo - drone should take off and fly!

This response is getting very long. Would you like me to:

1. **Continue with complete code for all remaining services** (Command Parser with NLP, Object Detection with YOLO, RL Navigator, Feedback Service)
2. **Create the complete PDF document with all this content** formatted properly
3. **Focus on specific parts** you want to see first

I have complete implementations ready for:

- ✅ Telemetry Collector (done above)
- ✅ Flight Control (done above)
- ⏳ Command Parser (NLP AI) - ready to write
- ⏳ Object Detection (YOLO) - ready to write
- ⏳ RL Navigator (Reinforcement Learning) - ready to write
- ⏳ Feedback Service - ready to write
- ⏳ Integration & Testing scripts - ready to write
- ⏳ Grafana dashboard setup - ready to write
- ⏳ Complete demo scenario - ready to write

Please let me know how you'd like me to proceed!